BUILD CITY × NEAR

[ BUILD CITY HACKPACK ] · ICM STARTER GUIDE

# BUILD WITH FOLDERS

From your one-line idea to a working ICM workspace, on any agent, with private inference on NEAR AI Cloud

icm-architect skill

## 00 / ICM INTERPRETABLE CONTEXT METHODOLOGY

4 pages · start to finish

01 / WHAT IT IS

## The folder is the agent's architecture

**ICM** (Interpretable Context Methodology) is a way to build AI workflows out of plain folders and text files instead of agent-framework code. **Numbered folders are the steps**, **markdown files carry the instructions and context**, and small scripts do the non-AI work. **One agent** reads the right files at each step, and **you review** between steps. State is just files, so anyone can open a folder and see where the work stands.

Created by Jake Van Clief & David McDermott, "Interpretable Context Methodology: Folder Structure as Agent Architecture" (arXiv:2603.16021, March 2026). Their claim, for step-by-step work a person reviews: **the folder structure can replace the agent framework.**

02 / WHY IT'S WORTH IT

## What you get from building this way

- **Every step leaves a file you can open and edit.** A mistake fixed in a text file before the expensive step costs seconds; found at the end, it costs a redo.
- **Load only what the step needs.** About 2k–8k tokens per step instead of ~42k in one giant prompt (paper estimate). Cheaper, and models do worse when the key facts are buried in long context.
- **Glass box.** Every in-between result is readable, so you can see exactly where a run went wrong.
- **Portable.** It's a folder: copy it, zip it, put it in Git, hand it off, or swap the agent and the model. The folder is the memory.
- **Anyone can change it.** People who don't code edit a `CONTEXT.md` file, not a program.
- **Cheaper on NEAR.** NEAR's TEE models cache a byte-identical prompt start, so a small, stable `AGENTS.md` read first every run is billed at the cheaper cache-read rate.

03 / ANOTHER WAY TO BUILD

## Prove the workflow in folders, then build the app

- **Folders instead of a framework.** Most "multi-agent" setups are really written instructions plus a map that routes to them. ICM keeps the instructions and the map as files, and keeps one agent. Add a second agent only when you can name the job it removes.
- **Test the automation before you build the app.** Run the workflow as folders first and watch which steps come out the same every time. A step done the same way three or four times becomes a script. Once the steps are proven, the stage contracts are the spec for an app. Our advice: ICM is the prototype; the app is the product.
- **Keep your judgment where it changes the result.** Each step stops at a check, and the AI waits for you there. You stay accountable for what ships, and a wrong turn is caught before the next step multiplies it.

| PICK | WHEN |
|---|---|
| **ICM** | Sequential work a person reviews; reordering, editing, inspecting and handing off steps; prototyping a workflow before you code it |
| **A framework or code** | Real-time teams of agents, many users at once, workflows that must branch on their own, automatic retries, scheduling and deployment |

Honest limits: the paper's evidence is early (informal, self-reported, a self-selected community). ICM has no model router, retries or deployment built in; you add those with a script or a tool like LiteLLM.

04 / START FROM YOUR ONE-LINE IDEA

## Your idea worksheet is the brief for your workspace

Take the **Final line** and the **feedback** you wrote on the idea worksheet (the back of the agenda sheet). Open an empty project folder in your agent (page 3 lists them; page 4 walks through Hermes) and give it this prompt:

**PROMPT · PASTE INTO YOUR AGENT, INSIDE THE EMPTY PROJECT FOLDER**

Use the icm-architect skill in Build mode. Build me an ICM workspace for my hackathon project: [your final line]. Feedback I got: [notes]. Ask me your questions a few at a time, then pick the form and scaffold the smallest workspace that carries the work: AGENTS.md as the entry file, numbered stages, and a CONTEXT.md contract in every stage. Model calls go to NEAR AI Cloud, and any stage that touches private data runs on a Confidential TEE model. Stop after the scaffold so I can review it before you build anything else.

### What the skill will ask you (have answers ready)

- **The repeating unit:** what one run works on (a client, a report, a case, a person?)
- **One run, start to finish:** where do you stop and check before going on?
- **Same every run vs. new every run:** rules, voice, schema vs. this run's inputs
- **What "done" looks like:** the file that leaves the workspace
- **Who else touches it,** and what they need to find without asking you
- Your pauses become stage boundaries; your "I always check X before Y" becomes a check.

05 / WHAT YOU'LL GET

## A small, readable workspace

```
my-project/
├── AGENTS.md          the map: where things live,
│                      where to go for each job
├── CLAUDE.md          one-line pointer to AGENTS.md
├── CONTEXT.md         the pipeline in one table
├── _shared/           stable rules, facts, voice
├── stages/
│   ├── 01_intake/     CONTEXT.md + output/
│   ├── 02_draft/      CONTEXT.md + output/
│   └── 03_review/     CONTEXT.md + output/
├── _templates/        blank starters you copy
└── _tools/            scripts (links.py checks links)
```

**EXAMPLE NAMES** Your stages come from your answers above.

### Every stage has a contract

| | |
|---|---|
| **Inputs** | exact file paths: this run's working files and the stable references |
| **Process** | the numbered steps |
| **Outputs** | what it writes, and where |
| **Check** | a person, a script or a small model; nothing moves on until it passes |

### Rules of thumb (from the skill)

- **One folder, one job,** and it says so in a file inside it
- **A small, stable entry file:** `AGENTS.md` under ~60 lines; it routes, it doesn't hold content
- **Numbers encode order:** `01_`, `02_`, `03_`
- **Every step ends at a check**
- **Load only what the step needs:** one contract plus its inputs under ~8,000 tokens
- **Reference apart from product:** rules in `_shared/`, run results in `output/`
- **Status lives on disk:** what exists is where it stands
- **Start small:** three real stages beat seven imagined ones

THE WALK TEST · BEFORE YOU BUILD ON IT

- Open the root cold: can you tell **where you are** and **where to go** for the current task from `AGENTS.md` plus at most two more files?
- Does every stage's `CONTEXT.md` name exact input paths, the job, the output and the check? Is any fact stored in two places? Pick one home and link to it.
- Run `python3 -I _tools/links.py` (if the skill copied it) to check that every link and input path resolves.



Skill: **github.com/RinDig/icm-architect** (MIT)
Method: Van Clief & McDermott, arXiv:2603.16021

06 / WORKS WITH ANY AGENT THAT CAN READ YOUR FOLDER

## Bring your own agent harness

ICM depends on no one vendor. It runs in any **agent harness**, meaning any AI coding agent or agent runtime that can **read and write files in a folder**: Claude Code, Codex, Gemini CLI, Hermes Agent, Cursor, GitHub Copilot, OpenCode and others. The model behind it is your choice too. Point the agent at NEAR AI Cloud and the model call runs privately in a TEE.

The workspace's entry file is `AGENTS.md`. Agents that look for another name (`CLAUDE.md`, `GEMINI.md`) get a one-line pointer to it, never a second copy, so every agent reads the same map.

INSTALL THE ICM-ARCHITECT SKILL

## One skill, every agent

The icm-architect skill teaches your agent the method: the invariants, the forms, the templates and the checks. It uses the open Agent Skills format (agentskills.io), so any agent that reads `SKILL.md` can use it. **Keep the folder named `icm-architect`**: a skill's name must match its folder.

**GET IT**

```
git clone https://github.com/RinDig/icm-architect.git
```

| AGENT | PUT THE `ICM-ARCHITECT/` FOLDER IN |
|---|---|
| **Claude Code** | `~/.claude/skills/` (all projects) or `.claude/skills/` in a project |
| **Codex** | `~/.agents/skills/` (all projects) or `.agents/skills/` in a repo |
| **Gemini CLI** | `~/.gemini/skills/` or `~/.agents/skills/`, or run `gemini skills install` with the repo's Git URL |
| **Hermes Agent** | `~/.hermes/skills/`, or list `~/.agents/skills` under `skills.external_dirs` in its config |
| **Cursor, GitHub Copilot** | that client's skills folder (same for any other Agent Skills client) |
| **Claude apps** | zip the folder (the folder itself is the zip root), then upload it under Customize → Skills |
| **Anything else** | give it the folder and say "use this skill", or paste `SKILL.md` and then each file it points to |

One copy in `~/.agents/skills/` serves Codex and Gemini CLI, and Hermes can be pointed at it. The skill's scripts need only Python 3's standard library; an agent without a shell can skip them.

THEN, IN ANY AGENT

## Three ways to use it

| | |
|---|---|
| **Build** | from a described process, idea or problem: "Build me a workspace for X", "ICM this" (start with the page 2 prompt) |
| **Restructure** | an existing folder or repo: "Structure this for agents", "Audit this folder". It inventories first, proposes a migration map for your OK, then copies, checks and removes |
| **Check** | "Run the walk test on this workspace", or run `python3 -I _tools/links.py` |

ROUTE BY PRIVACY · OUR ADVICE

- A stage that touches private data names a **Confidential TEE** model in its `CONTEXT.md` (for example `model: z-ai/glm-5.3-flash`). A stage with only public data may use any model. Writing the model in the contract puts the choice where a person can see it.
- Only the model call is private. Your agent, your laptop and the files it sends sit outside the TEE.



Live model list: **curl https://cloud-api.near.ai/v1/models**
Checked for this guide: Nov 10, 2026

07 / BUILD IT END TO END WITH HERMES ON NEAR AI CLOUD

# Hermes Agent + private inference, from zero to your first stage

Hermes Agent (Nous Research) is an open-source, model-agnostic agent that runs from the terminal and reads `AGENTS.md` in its working folder automatically. Tested on a Mac with Hermes v0.22.1 (2026-10-14). Works on macOS, Linux and WSL2.

## 1 Get your NEAR AI Cloud key

Claim $10 in credits at **nearbuilders.org/dashboard** (watch "Claim AI Credits" in the hackpack); the key starts with `sk-` and is shown once. Or make one at cloud.near.ai → API Keys. **Never paste it into a chat, show it on screen or commit it.**

## 2 Install Hermes, then open a new terminal

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

The command lands in `~/.local/bin/hermes`; settings in `~/.hermes/config.yaml`; keys in `~/.hermes/.env`.

## 3 Put your key in `~/.hermes/.env` yourself, as one line

```
NEAR_API_KEY=sk-your-key-here
```

## 4 Point Hermes at NEAR, and use only NEAR: edit `~/.hermes/config.yaml`

```
model:
  default: "z-ai/glm-5.3-flash"
  provider: "custom"
  base_url: "https://cloud-api.near.ai/v1"
  key_env: NEAR_API_KEY
auth:
  adopt_external_logins: false
```

`key_env` keeps the key out of the config file. `z-ai/glm-5.3-flash` is NEAR's own Confidential TEE model with tools and reasoning (on the live list, Nov 10). `adopt_external_logins: false` stops Hermes borrowing your Claude Code or Codex login, so every answer really comes from NEAR.

## 5 Test it

```
hermes -z "Say hello in one sentence."    # one answer, then exit
hermes config get model                    # model, provider, URL (never the key)
```

## 6 Add the icm-architect skill

```
git clone https://github.com/RinDig/icm-architect.git ~/.hermes/skills/icm-architect
```

## 7 Make your project folder and start Hermes inside it

```
mkdir my-project && cd my-project && git init
hermes
```

Open the folder in VS Code and run `hermes` in its terminal, so you see the files appear. Paste the **page 2 prompt** with your final line.

## 8 Review at every stop, then run stage by stage

Read the scaffold, edit anything that's wrong, then say go. `git commit` before each stage so you can roll back. After you change `AGENTS.md`, run `hermes --exit` and start a new session: Hermes reads it only at start.

## 9 Ship it to the NEAR Hackathon

Make the repo public with a README that says what it does. Hand **nearhackathon.com/HACKATHON.md** to your agent for the submission steps; you choose the tracks, review the payload and confirm before it submits, and you post the X demo yourself. Deadline: **Sat Nov 14, 6:59 PM ET**.

### WATCH OUT

- **Only the model call is private.** Hermes, your laptop, the files it reads and its browser, web and messaging tools are outside the TEE. It has read files outside its folder (like `~/Downloads`): keep private files elsewhere.
- **It may write files without asking.** Work in Git and keep the stops in your `AGENTS.md`. **Slow answers?** Thinking is on by default (about 21–26 tokens/s in our tests); turn it off with `chat_template_kwargs: {"enable_thinking": false}`.



Stuck? NEAR Builders Telegram **@nearbuilderschat** (make a Telegram account first)
Learn ICM free: **Clief Notes** community on Skool